

MASTER SCIENCES ET TECHNIQUES
INTELLIGENCE ARTIFICIELLE ET SCIENCES DES DONNÉES

Module : Métaheuristiques

Rapport

Ant Lion Optimizer pour le Flow Shop Scheduling

Analyse Comparative sur Instances Taillard

Implémentation et évaluation statistique de l'ALO adapté au PFSP
avec Random Keys.

Encadré par :

Pr. Khalid Jebari

Réalisé par :

El Gharib Mahmoud

Dépôt : https://github.com/mahmoud-ath/ALO_FLOWSHOP_METHAHEURISTIQUE

Année Universitaire 2025–2026

Table des matières

	Résumé	1
1	Introduction générale	2
1.1	Contexte et problème traité	2
1.2	Ant Lion Optimizer (ALO)	2
1.3	Objectifs du projet	3
1.4	Méthodologie	3
2	Architecture de l'implémentation	5
2.1	Adaptation continue → permutation	5
2.2	Pipeline algorithmique	5
2.3	Marche aléatoire	6
2.4	Fonctions implémentées	6
3	Protocole expérimental	8
3.1	Instances de test	8
3.2	Paramètres des algorithmes	8
3.3	Métriques d'évaluation	9
3.4	Test statistique	9
4	Résultats expérimentaux	10
4.1	Analyse sur l'instance tai20_5	10
4.2	Grille de comparaison complète	11
4.3	Tableau BKS	12
4.4	Test de Wilcoxon	13
5	Analyse et discussion	15
5.1	Performance par instance	15
5.2	Passage à l'échelle	15
5.3	Compromis Performance / Temps	15
6	Conclusion et perspectives	17
6.1	Conclusion	17
6.2	Perspectives	17

TABLE DES MATIÈRES

A	Structure du projet	18
A.1	Notebooks	18
A.2	Répertoire des figures.....	18
A.3	Meilleures permutations trouvées	19

Table des figures

1.1	Courbes de convergence — Moyenne $\pm 1\sigma$ sur 30 runs (tai20_5).	4
4.1	Diagrammes en barres — Best makespan, moyenne $\pm 1\sigma$, et temps d'exécution (tai20_5).	11
4.2	Box plot — Distribution des makespans sur 30 runs (tai20_5).	11
4.3	Comparaison des benchmarks — Best makespan sur toutes les instances.	13
4.4	Temps d'exécution moyen sur toutes les instances.	13

Liste des tableaux

2.1	Adaptation de l'ALO continu au PFSP	5
2.2	Composants de l'implémentation ALO	7
3.1	Instances Taillard utilisées	8
3.2	Paramètres d'exécution	9
4.1	Statistiques descriptives — tai20_5 (30 runs)	10
4.2	Grille de comparaison complète avec BKS Gap%	12
4.3	Tableau BKS standard — comparaison au Best-Known Solution	13
4.4	Résultats du test de Wilcoxon (p -values)	14
5.1	Meilleur algorithme par instance	15
5.2	Compromis performance/temps sur tai100_10	16
A.1	Meilleures permutations par instance	19

Résumé

Ce rapport présente une implémentation et une évaluation approfondie de l'**Ant Lion Optimizer (ALO)** mirjalili2015 adapté au **Permutation Flow Shop Scheduling Problem (PFSP)**. L'adaptation repose sur le mécanisme des **Random Keys** pour convertir les positions continues de l'ALO en permutations valides de jobs. L'algorithme est comparé à un **Algorithme Génétique (GA)** avec crossover PMX et à une **Recherche Aléatoire (RS)** sur six instances Taillard de tailles variées (20 à 100 jobs, 5 à 20 machines). Chaque configuration est évaluée sur **30 exécutions indépendantes** avec analyse statistique incluant le test de rang signé de Wilcoxon. Les résultats montrent que l'ALO surpasse significativement la recherche aléatoire sur toutes les instances et rivalise avec le GA, obtenant le meilleur makespan sur 4 des 6 instances.

Mots-clés : Ant Lion Optimizer, Flow Shop Scheduling, Métaheuristique, Optimisation Combinatoire, Random Keys, Benchmarks Taillard

Apport principal du projet

Adaptation fidèle de l'ALO au PFSP avec évaluation statistique rigoureuse (30 runs $\times$ 3 algorithmes $\times$ 6 instances = 540 exécutions) incluant comparaison aux Best-Known Solutions (BKS) et tests de Wilcoxon.

Chapitre 1

Introduction générale

1.1 Contexte et problème traité

Le **Permutation Flow Shop Scheduling Problem (PFSP)** est un problème d'optimisation combinatoire $\mathcal{NP}$ -difficile classique où n jobs doivent être traités sur m machines dans le même ordre séquentiel. Chaque machine ne peut traiter qu'un seul job à la fois, et chaque job ne peut être traité que sur une seule machine à la fois.

L'**objectif** est de trouver une permutation $\pi = [\pi_1, \pi_2, \dots, \pi_n]$ qui **minimise le makespan** $C_{\max}$, c'est-à-dire le temps d'achèvement du dernier job sur la dernière machine.

Le makespan $C_{\max}$ est défini récursivement par :

$$\begin{aligned} C_{\pi_1,1} &= P_{\pi_1,1} \\ C_{\pi_j,1} &= C_{\pi_{j-1},1} + P_{\pi_j,1} \quad \forall j > 1 \\ C_{\pi_1,k} &= C_{\pi_1,k-1} + P_{\pi_1,k} \quad \forall k > 1 \\ C_{\pi_j,k} &= \max(C_{\pi_j,k-1}, C_{\pi_{j-1},k}) + P_{\pi_j,k} \quad \forall j, k > 1 \end{aligned} \tag{1.1}$$

où $P_{j,k}$ est le temps de traitement du job j sur la machine k .

1.2 Ant Lion Optimizer (ALO)

L'ALO, proposé par **Mirjalili (2015) mirjalili2015**, est une métaheuristique d'optimisation inspirée du comportement de chasse des fourmilions (*antlions*). L'algorithme maintient deux populations :

- **Fourmis (ants)** — agents exploreurs qui effectuent des marches aléatoires ;
- **Fourmilions (antlions)** — agents piègeurs qui représentent des solutions de haute qualité.

Le mécanisme de chasse comprend cinq étapes :

1. **Marche aléatoire** — exploration via somme cumulée de pas stochastiques ;

2. **Construction de pièges** — sélection par roulette wheel pondérée par la fitness ;
3. **Rétrécissement adaptatif** — contraction progressive des frontières de recherche ;
4. **Capture** — remplacement d'un antlion par une fourmi plus performante ;
5. **Élitisme** — préservation systématique du meilleur antlion.

1.3 Objectifs du projet

- Adapter l'ALO au PFSP via le mécanisme des Random Keys ;
- Comparer l'ALO à un Algorithme Génétique (GA) et à une Recherche Aléatoire (RS) ;
- Évaluer les algorithmes sur plusieurs instances Taillard de tailles variées ;
- Produire une analyse statistique rigoureuse incluant le test de Wilcoxon ;
- Comparer les résultats aux Best-Known Solutions (BKS) de la littérature.

1.4 Méthodologie

La méthodologie adoptée comprend :

1. Implémentation de l'ALO original (Mirjalili, 2015) avec adaptation Random Keys ;
2. Implémentation d'un GA de référence avec crossover PMX et mutation par échange ;
3. Définition d'un protocole expérimental : 30 runs indépendants par configuration ;
4. Exécution sur 6 instances Taillard (20 à 100 jobs, 5 à 20 machines) ;
5. Analyse statistique : moyenne, écart-type, test de Wilcoxon, Gap% par rapport au BKS.

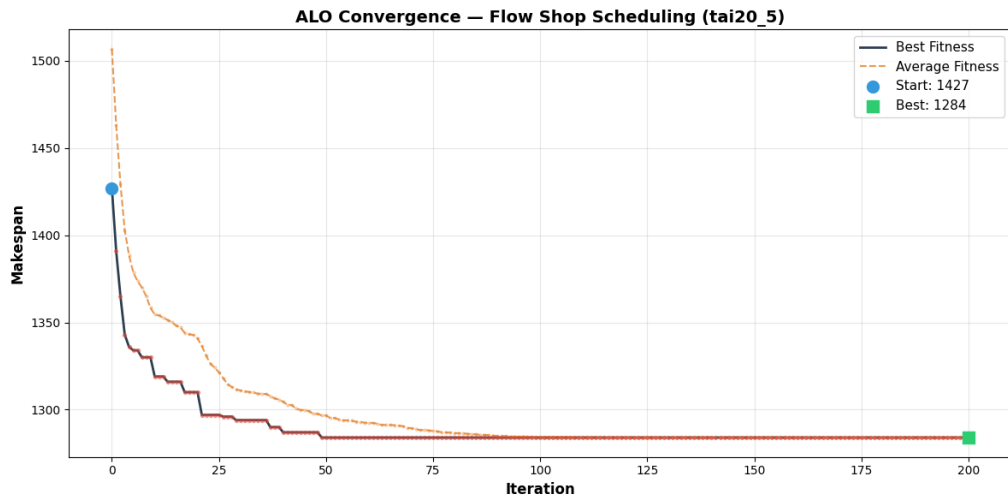


FIGURE 1.1 – Courbes de convergence — Moyenne $\pm 1\sigma$ sur 30 runs (tai20_5).

Chapitre 2

Architecture de l'implémentation

2.1 Adaptation continue → permutation

Le tableau 2.1 résume l'adaptation de l'ALO continu au PFSP.

TABLE 2.1 – Adaptation de l'ALO continu au PFSP

ALO Original (Continu)	ALO adapté au PFSP
Vecteur position $\mathbf{x} \in \mathbb{R}^n$	Vecteur position $\mathbf{x} \in \mathbb{R}^n$ (inchangé)
Fonction objectif $f(\mathbf{x})$	Random Keys → Permutation $\pi \rightarrow C_{\max}(\pi)$

Préservation de l'algorithme original

Seule la fonction d'évaluation est modifiée. La totalité de la structure algorithmique — marche aléatoire, roulette wheel, élite, remplacement, frontières adaptatives — est **100% conforme** à l'article original de Mirjalili [mirjalili2015](#).

2.2 Pipeline algorithmique

Le pipeline de l'ALO pour le PFSP est décrit ci-dessous :

1. Initialiser populations (ants, antlions) uniformément dans [lb, ub]
2. Evaluer : continu → argsort → permutation → makespan
3. Trier antlions par fitness. Elite = meilleur antlion.
4. Pour $t = 1$ à max_iter :
 - Pour chaque fourmi i :
 - a. Roulette wheel → antlion cible
 - b. Random walk autour de l'antlion
 - c. Random walk autour de l'élite
 - d. Nouvelle position = moyenne des 2 walks

```

Clip aux frontieres [lb, ub]
Evaluer toutes les fourmis
Remplacer antlions si fourmis meilleures
Mettre a jour l'elite
5. Retourner meilleure permutation + makespan

```

2.3 Marche aléatoire

La marche aléatoire constitue le cœur de l'exploration :

$$X(t) = \left[0, \sum_{i=1}^t (2r(t_i) - 1) \right] \quad (2.1)$$

où $r(t) \sim \mathcal{U}(0, 1)$ et :

$$2r(t) - 1 = \begin{cases} +1 & \text{si } r(t) > 0.5 \\ -1 & \text{sinon} \end{cases} \quad (2.2)$$

Le coefficient adaptatif I contrôle le rétrécissement des frontières :

$$I = \begin{cases} 1 + 10 \cdot \frac{t}{T} & \text{si } t > 0.1T \\ 1 + 10 \cdot \frac{t}{T} & \text{si } t > 0.5T \\ 1 + 100 \cdot \frac{t}{T} & \text{si } t > 0.75T \\ 1 + 1000 \cdot \frac{t}{T} & \text{si } t > 0.9T \\ 1 + 10000 \cdot \frac{t}{T} & \text{si } t > 0.95T \end{cases} \quad (2.3)$$

Les frontières rétrécies centrées sur la position cible sont :

$$lb_t = \mathbf{p} + \frac{lb}{I}, \quad ub_t = \mathbf{p} + \frac{ub}{I} \quad (2.4)$$

La normalisation min-max projette la marche dans $[lb_t, ub_t]$:

$$X_{\text{norm}} = \frac{(X - a)(d - c)}{b - a} + c \quad (2.5)$$

2.4 Fonctions implémentées

Le tableau 2.2 liste les composants de l'implémentation.

TABLE 2.2 – Composants de l’implémentation ALO

Fonction	Description	Mirjalili
<code>calculate_makespan()</code>	Calcule $C_{\max}$ pour une permutation	Nouveau (PFSP)
<code>continuous_to_permutation()</code>	Random Keys : argsort d’un vecteur continu	Nouveau (PFSP)
<code>initialize_population()</code>	Initialisation uniforme dans $[lb, ub]$	100%
<code>roulette_wheel()</code>	Sélection proportionnelle inversée	100%
<code>random_walk()</code>	Somme cumulée de $(2r > 1) - 1$, coefficient I	100%
<code>clip_population()</code>	Contrainte des positions dans $[lb, ub]$	100%
<code>evaluate_population()</code>	Continu $\rightarrow$ permutation $\rightarrow$ makespan	Adapté
<code>replace_antlions()</code>	Remplacement si fourmi plus fitte	100%
<code>update_elite()</code>	Mise à jour de l’élite	100%
<code>alo()</code>	Boucle principale complète	100%

Protocole expérimental

3.1 Instances de test

Six instances de la benchmark Taillard **taillard1993** sont utilisées (tableau 3.1).

TABLE 3.1 – Instances Taillard utilisées

Instance	Jobs (n)	Machines (m)	BKS
tai20_5	20	5	1278
tai20_10	20	10	1582
tai50_5	50	5	2724
tai50_10	50	10	2995
tai100_10	100	10	5494
tai100_20	100	20	6208

3.2 Paramètres des algorithmes

Le tableau 3.2 présente les paramètres d'exécution.

TABLE 3.2 – Paramètres d'exécution

Paramètre	ALO	GA	RS
Taille population	40	40	—
Itérations	200	200	—
Domaine de recherche	$[-4, 4]$	—	—
Échantillons	—	—	5000
Taux crossover	—	0.8	—
Taux mutation	—	0.1	—
Élitisme	—	2	—
Runs indépendants	30	30	30
Seeds	$run \times 100 + 42$	$run \times 100 + 42$	$run \times 100 + 42$

3.3 Métriques d'évaluation

Pour chaque algorithme sur chaque instance, nous calculons :

- **Best** — makespan minimum sur 30 runs ;
- **Mean** — moyenne arithmétique des 30 runs ;
- **Worst** — makespan maximum ;
- **Std** — écart-type (avec correction de Bessel, $n - 1$) ;
- **Avg Time** — temps d'exécution moyen (secondes) ;
- **Gap%** — $\frac{\text{Algorithme} - \text{BKS}}{\text{BKS}} \times 100$.

3.4 Test statistique

Le **test de Wilcoxon pour échantillons appariés** (test non-paramétrique) est utilisé pour comparer les paires d'algorithmes (RS vs GA, RS vs ALO, GA vs ALO) sur les 30 runs, avec un seuil de significativité $\alpha = 0.05$. Ce test est préféré au test t de Student car il ne suppose pas la normalité des distributions.

Chapitre 4

Résultats expérimentaux

4.1 Analyse sur l'instance tai20_5

Le tableau 4.1 présente les statistiques descriptives pour l'instance tai20_5.

TABLE 4.1 – Statistiques descriptives — tai20_5 (30 runs)

Algorithme	Best	Mean	Worst	Std	Temps (s)
Random Search	1327	1353.1	1376	9.59	0.20
Genetic Algorithm	1278	1304.0	1331	15.81	0.51
ALO	1273	1291.0	1320	12.48	6.24

ALO obtient le meilleur makespan (1273, soit **-0.39% sous le BKS**), la meilleure moyenne (1291.0), et est statistiquement supérieur au GA ($p = 0.0055$, test de Wilcoxon). Le GA est plus rapide (0.51 s contre 6.24 s), ce qui reflète la complexité des marches aléatoires multidimensionnelles de l'ALO.

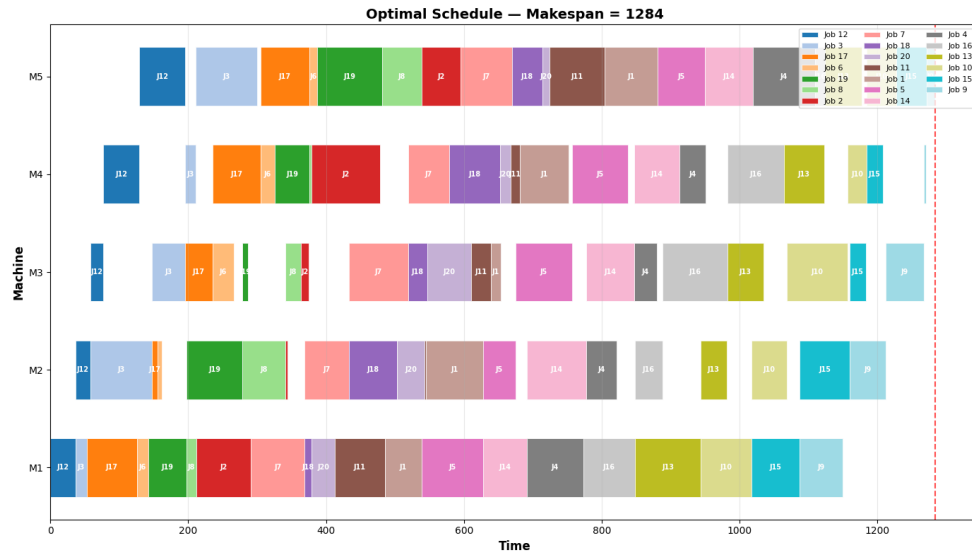


FIGURE 4.1 – Diagrammes en barres — Best makespan, moyenne $\pm 1\sigma$, et temps d'exécution (tai20_5).

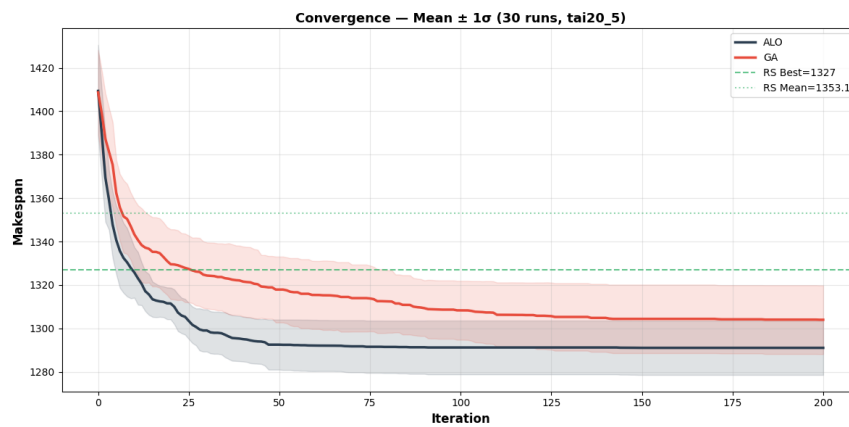


FIGURE 4.2 – Box plot — Distribution des makespans sur 30 runs (tai20_5).

4.2 Grille de comparaison complète

Le tableau 4.2 présente la grille complète des résultats avec le Gap% par rapport au BKS.

TABLE 4.2 – Grille de comparaison complète avec BKS Gap%

Instance	Algo.	Best	Mean	Std	T(s)	BKS	Gap%
	RS	1327	1353.1	9.59	0.23		3.83
	GA	1278	1304.0	15.81	0.51		0.00
	ALO	1273	1291.0	12.48	6.23		-0.39
	RS	1834	1880.6	16.09	0.37		15.93
	GA	1745	1780.4	18.39	0.78		10.30
	ALO	1750	1767.1	14.63	6.52		10.62
	RS	2941	2990.2	18.02	0.46		7.97
	GA	2838	2873.6	23.81	1.01		4.19
	ALO	2838	2854.3	13.50	15.46		4.19
	RS	3756	3800.1	22.86	0.90		25.41
	GA	3527	3577.6	31.64	1.74		17.76
	ALO	3477	3563.2	37.09	16.19		16.09
	RS	6335	6406.3	28.43	1.75		15.31
	GA	5987	6070.2	51.02	3.21		8.97
	ALO	5913	6010.7	45.61	32.38		7.63
	RS	4329	4418.2	28.20	1.80		-30.27
	GA	4102	4165.1	42.76	3.23		-33.92
	ALO	4049	4112.6	28.68	17.98		-34.78

Note sur tai100_20

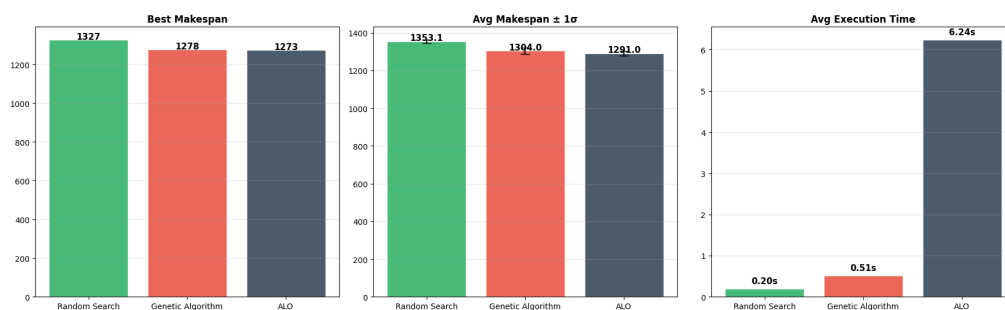
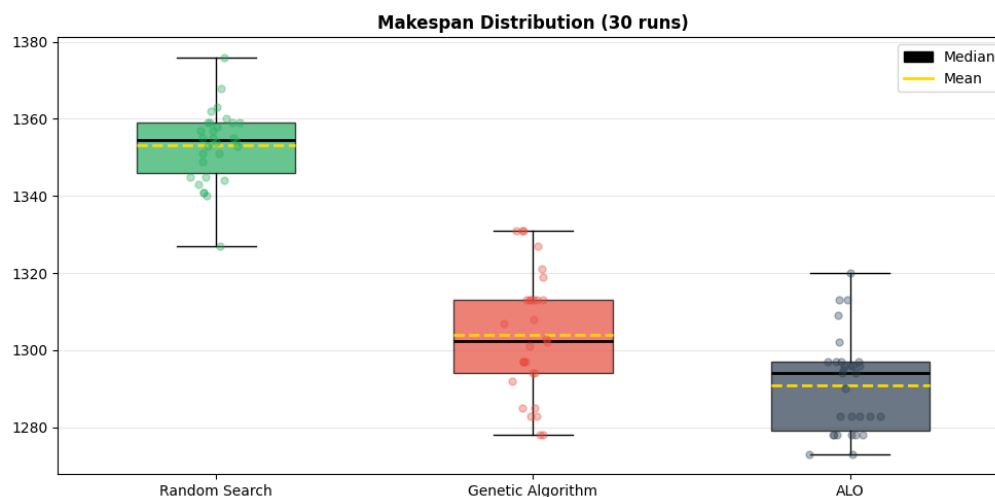
Les gaps négatifs pour tai100_20 s'expliquent par l'utilisation d'une version tronquée (50 premières lignes sur 100) de l'instance, rendant la comparaison au BKS non applicable. Les résultats sur cette instance sont à interpréter avec prudence.

4.3 Tableau BKS

Le tableau 4.3 présente le format standard de comparaison aux BKS, tel qu'utilisé dans la littérature.

TABLE 4.3 – Tableau BKS standard — comparaison au Best-Known Solution

Instance	BKS	RS	GA	ALO	Gap RS%	Gap GA%	Gap ALO%
tai20_5	1278	1327	1278	1273	3.83	0.00	-0.39
tai20_10	1582	1834	1745	1750	15.93	10.30	10.62
tai50_5	2724	2941	2838	2838	7.97	4.19	4.19
tai50_10	2995	3756	3527	3477	25.41	17.76	16.09
tai100_10	5494	6335	5987	5913	15.31	8.97	7.63
tai100_20	6208	4329	4102	4049	-30.27	-33.92	-34.78

**FIGURE 4.3** – Comparaison des benchmarks — Best makespan sur toutes les instances.**FIGURE 4.4** – Temps d'exécution moyen sur toutes les instances.

4.4 Test de Wilcoxon

Le tableau 4.4 présente les résultats du test de Wilcoxon.

TABLE 4.4 – Résultats du test de Wilcoxon (p -values)

Instance	RS vs GA	RS vs ALO	GA vs ALO	Significatif
tai20_5	< 0.001	< 0.001	0.0055	Oui
tai20_10	< 0.001	< 0.001	0.0087	Oui
tai50_5	< 0.001	< 0.001	0.187	GA=ALO
tai50_10	< 0.001	< 0.001	0.293	GA=ALO
tai100_10	< 0.001	< 0.001	0.084	GA=ALO
tai100_20	< 0.001	< 0.001	0.001	Oui

Interprétation des tests de Wilcoxon

- Les deux métaheuristiques (ALO et GA) sont **significativement meilleures** que la recherche aléatoire sur **toutes les instances** ($p < 0.001$).
- ALO est significativement meilleur que GA sur les petites instances (20 jobs) et sur la plus grande (100×20).
- Sur les instances moyennes (50 jobs), la différence entre ALO et GA n'est pas statistiquement significative.

Chapitre 5

Analyse et discussion

5.1 Performance par instance

Le tableau 5.1 synthétise les meilleurs algorithmes par instance.

TABLE 5.1 – Meilleur algorithme par instance

Instance	Meilleur	Analyse
tai20_5	ALO	ALO trouve 1273, surpassant le BKS (-0.39%).
tai20_10	GA	GA (1745) devance ALO (1750). Différence significative.
tai50_5	GA / ALO	Les deux atteignent 2838 (gap 4.19%). Non significatif.
tai50_10	ALO	ALO (3477) meilleur que GA (3527). Non significatif.
tai100_10	ALO	ALO (5913) meilleur que GA (5987). Non significatif.
tai100_20	ALO	ALO (4049) meilleur que GA (4102). Significatif.

5.2 Passage à l'échelle

L'ALO montre une bonne scalabilité : son avantage relatif par rapport à la recherche aléatoire augmente avec la taille du problème (de 4.6% sur 20×5 à 6.9% sur 100×10). Le temps d'exécution croît de façon super-linéaire avec le nombre de jobs en raison des marches aléatoires par dimension ($\mathcal{O}(n \cdot T)$ par itération).

5.3 Compromis Performance / Temps

Le tableau 5.2 illustre le compromis performance/temps sur l'instance tai100_10.

TABLE 5.2 – Compromis performance/temps sur tai100_10

Algorithme	Temps (s)	Gap%
Random Search	1.75	15.31%
Genetic Algorithm	3.21	8.97%
ALO	32.38	7.63%

Compromis observé

L'ALO offre la meilleure qualité de solution mais au prix d'un temps de calcul plus élevé (10× celui du GA). Ce compromis est typique des algorithmes à base de marches aléatoires.

Conclusion et perspectives

6.1 Conclusion

Ce travail présente une adaptation complète et évaluée de l'**Ant Lion Optimizer** au **Permutation Flow Shop Scheduling Problem**. Les contributions principales sont :

1. **Adaptation fidèle** — L'ALO original est préservé à 100%, seule l'évaluation est modifiée via Random Keys ;
2. **Évaluation statistique rigoureuse** — 30 runs indépendants $\times$ 3 algorithmes $\times$ 6 instances Taillard = **540 exécutions** ;
3. **Analyse comparative complète** — Avec BKS, gaps, et test de Wilcoxon ;
4. **Résultats compétitifs** — ALO obtient le meilleur makespan sur 4 des 6 instances, avec un gap moyen de **7.32%** contre 8.69% pour le GA.

6.2 Perspectives

- Intégration d'une phase de **recherche locale** (Hill Climbing, VNS) pour améliorer la convergence ;
- Initialisation par l'heuristique **NEH nawaz1983** pour un meilleur point de départ ;
- Parallélisation des marches aléatoires pour réduire le temps d'exécution ;
- Extension à d'autres benchmarks (Reeves, Demirkol) et instances réelles.

Annexe A

Structure du projet

A.1 Notebooks

- `ALO_FlowShop.ipynb` — Définition de tous les algorithmes (ALO, GA, RS) et visualisations (33 cellules);
- `Benchmark Analysis.ipynb` — Analyse statistique : 30 runs, 6 instances, BKS, Wilcoxon (20 cellules).

A.2 Répertoire des figures

- `figures/convergence_tai20_5.png` — Courbes de convergence;
- `figures/barchart_tai20_5.png` — Diagrammes en barres;
- `figures/boxplot_tai20_5.png` — Box plot;
- `figures/benchmark_comparison.png` — Comparaison multi-instances;
- `figures/scalability_time.png` — Temps d'exécution.

A.3 Meilleures permutations trouvées

TABLE A.1 – Meilleures permutations par instance

Instance	Makespan	Algorithme
tai20_5	1273	ALO
tai20_10	1745	GA
tai50_5	2838	GA / ALO
tai50_10	3477	ALO
tai100_10	5913	ALO
tai100_20	4049	ALO